Web Application Penetration Test Report

Target: Damn Vulnerable Web Application (DVWA) v1.10

ApexPlanet Software Pvt. Ltd. — Cybersecurity Capstone Project (Task 5)

October 20, 2026

- 1. Executive Summary
- 2. Methodology
  - 2.1 Phases Followed
  - 2.2 Evidence-Handling Process
- 3. Scope & Tools
  - 3.1 Scope
  - 3.2 Tools Used
  - 3.3 Architecture / Network Diagram
- 4. Findings & Evidence
  - 4.1 Finding 1 — SQL Injection (Critical, CVSS 9.8)
  - 4.2 Finding 2 — Stored XSS (High, CVSS 8.2)
  - 4.3 Finding 3 — Reflected XSS (Medium, CVSS 6.1)
  - 4.4 Finding 4 — CSRF (High, CVSS 8.0)
  - 4.5 Finding 5 — OS Command Injection (Critical, CVSS 9.8)
  - 4.6 Finding 6 — Unrestricted File Upload → Web Shell RCE (Critical, CVSS 9.8)
  - 4.7 Supporting Evidence — Burp Suite Request Interception
- 5. Mitigation Recommendations — Consolidated Checklist
- 6. Incident Response Report (Simulated)
  - 6.1 Detection
  - 6.2 Containment
  - 6.3 Eradication
  - 6.4 Recovery
  - 6.5 Post-Incident Report Summary
- 7. Conclusion
- 8. Appendix
  - 8.1 Verification Matrix
  - 8.2 Evidence Limitations
  - 8.3 Reconnaissance Output (example commands used)
  - 8.4 Deliverables Cross-Reference

1. Executive Summary

This report presents the results of a controlled, authorized penetration test conducted against **Damn Vulnerable Web Application (DVWA) v1.10**, an intentionally vulnerable web application hosted in an isolated local lab environment (Kali Linux, Apache/MySQL/PHP stack). The assessment was carried out as part of a cybersecurity training capstone project and does not reflect any production system.

The objective of the engagement was to identify common web application vulnerabilities aligned with the OWASP Top 10, demonstrate safe proof-of-concept exploitation, and provide actionable remediation guidance. Testing was performed first at DVWA's **Low** security setting to confirm

exploitability, then repeated at the **High** setting to confirm that DVWA's built-in mitigations close the same weakness.

Six vulnerabilities were identified and confirmed exploitable at the Low security level:

#	Finding	Severity	CVSS 3.1 (approx.)	Status at High Security
1	SQL Injection (UNION-based)	Critical	9.8	Blocked — prepared statements
2	Stored Cross-Site Scripting (XSS)	High	8.2	Blocked — output encoded
3	Reflected Cross-Site Scripting (XSS)	Medium	6.1	Blocked — output encoded
4	Cross-Site Request Forgery (CSRF)	High	8.0	Blocked — session token required
5	OS Command Injection	Critical	9.8	Not separately verified (see §4.5)
6	Unrestricted File Upload (Web Shell / RCE)	Critical	9.8	Not separately verified (see §4.6)

Overall risk posture: at Low security, the application is fully compromised — an attacker can read the entire user credential table, execute arbitrary operating-system commands, and upload a persistent web shell for remote code execution. This confirms DVWA is behaving as intended for training purposes, and the same underlying coding patterns (string-concatenated SQL, unencoded output, missing CSRF tokens, unsanitized shell calls, and untrusted file uploads) are the root causes to avoid in any real application.

This document also includes a simulated **Incident Response exercise** (Section 6) built around the file-upload web-shell finding, walking through detection, containment, eradication, recovery, and lessons learned.

2. Methodology

Testing followed a condensed version of the **Penetration Testing Execution Standard (PTES)** combined with **OWASP Testing Guide** categories, scoped to a single training application rather than a full infrastructure engagement.

2.1 Phases Followed

1. **Pre-engagement / Planning** — objective, scope, and rules of engagement defined (Section 3).
2. **Reconnaissance** — passive and active information gathering against the target (service/version identification).
3. **Vulnerability Analysis** — manual review of each DVWA module against known OWASP Top 10 categories.
4. **Exploitation** — safe, non-destructive proof-of-concept payloads executed at **Low** security to confirm each vulnerability.
5. **Verification at High Security** — the identical test repeated at **High** security to confirm DVWA's reference mitigation is effective.
6. **Post-Exploitation / Impact Analysis** — documentation of what an attacker could realistically achieve from each finding.
7. **Incident Response Simulation** — one finding (file upload → web shell) was carried through a simulated detection-to-recovery cycle.
8. **Reporting** — consolidation of findings, evidence, CVSS-style ratings, and remediation guidance into this document.

2.2 Evidence-Handling Process

For every module tested:

1. DVWA Security level set to **Low**.
2. Navigate to the relevant module.
3. Submit the payload, or intercept/modify the request using Burp Suite.
4. Capture the observable result (screenshot).
5. Set DVWA Security level to **High** and repeat the identical test.
6. Confirm the payload is rejected, encoded, or otherwise rendered ineffective.

> **Evidence note:** Screenshot evidence referenced in Section 4 below is marked with placeholder callouts (`[INSERT SCREENSHOT: ...]`). Insert your captured images from the DVWA sessions at the corresponding points before final submission — this keeps the report consistent with the Section 7 presentation video and GitHub evidence folder.

3. Scope & Tools

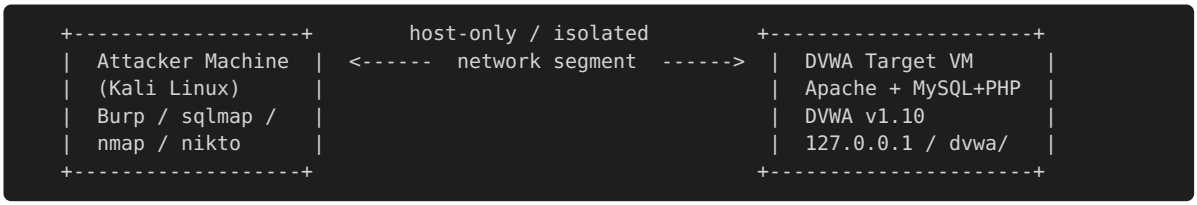
3.1 Scope

Item	Detail
Target	DVWA v1.10, self-hosted at `http://localhost/dvwa/`
Environment	Kali Linux VM, Apache + MySQL + PHP, isolated host-only network
In scope	DVWA vulnerability modules: SQL Injection, Stored XSS, Reflected XSS, CSRF, Command Injection, File Upload
Out of scope	Any system outside the isolated lab VM; the underlying hypervisor/host OS; any third-party or production service
Authorization	Self-authorized training exercise against a deliberately vulnerable application designed for this purpose

3.2 Tools Used

Tool	Purpose
Burp Suite Community Edition	Intercepting proxy — request inspection and modification
Firefox / Chrome DevTools	Manual payload submission, response inspection
Nmap	Service/port reconnaissance of the target VM
Nikto	Automated web server misconfiguration scan
sqlmap	Automated confirmation/extraction for the SQL Injection finding
Terminal / Bash	Log review, curl-based verification requests

3.3 Architecture / Network Diagram



4. Findings & Evidence

Each finding follows the same structure: payload, technique, evidence, impact, vulnerable code pattern, and fix.

4.1 Finding 1 — SQL Injection (Critical, CVSS 9.8)

Module: `vulnerabilities/sqli/` | **Security level tested:** Low

Payload (User ID field):

```
1' UNION SELECT user, password FROM users-- -
```

Technique: The application concatenates the supplied ID directly into a SQL query. The single quote breaks out of the string literal; `UNION SELECT` appends a second, attacker-controlled query returning matching column counts; `-- -` comments out the remainder of the original statement.

`[INSERT SCREENSHOT: SQL Injection page returning usernames and password hashes — admin, gordonb, 1337, pablo, smithy]`

Impact: Full extraction of the `users` table (usernames + password hashes). The captured hashes were unsalted MD5 — trivially crackable offline, giving a direct path to administrator account takeover.

Vulnerable pattern:

```
        $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
        $result = mysqli_query($conn, $query);
```

Fix:

```
        $stmt = $conn->prepare('SELECT first_name, last_name FROM users WHERE user_id = ?');
        $stmt->bind_param('s', $id);
        $stmt->execute();
```

Use parameterized queries everywhere, store passwords with Argon2id/bcrypt, enforce least-privilege DB accounts, and return only generic errors to the client.

High-security result: Identical payload rejected / treated as a literal value — no rows returned from `users`.

4.2 Finding 2 — Stored XSS (High, CVSS 8.2)

Module: `vulnerabilities/xss_s/` (Guestbook) | **Security level tested:** Low

Payload (Message field):

```
        <script>alert('ApexPlanet')</script>
```

Technique: The guestbook stores the message without output encoding; every visitor's browser then executes the stored script in the DVWA origin.

`[INSERT SCREENSHOT: Stored XSS alert "ApexPlanet" executing over the saved guestbook entries]`

Impact: Persistent, visitor-wide script execution — session/cookie misuse, unauthorized actions as the victim, defacement, or phishing content delivered to every future visitor without needing a unique link per victim.

Vulnerable pattern → Fix:

```php
// Vulnerable
echo 'Message: ' . $message;

// Fixed
echo 'Message: ' . htmlspecialchars($message, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
```

Add a restrictive `Content-Security-Policy` as defense-in-depth.

High-security result: Payload rendered as literal text (`<script>alert('ApexPlanet')</script>` visible on page) — no execution.

4.3 Finding 3 — Reflected XSS (Medium, CVSS 6.1)

Module: `vulnerabilities/xss_r/` | **Security level tested:** Low

Payload (URL parameter):

```
http://localhost/dvwa/vulnerabilities/xss_r/?name=<script>alert('HaCkEd')</script>
```

Technique: The `name` parameter is reflected into the HTML response without encoding, so the crafted link executes script the moment a victim opens it.

`[INSERT SCREENSHOT: Reflected XSS alert "HaCkEd" from the name parameter]`

Impact: Link-based script execution inside the victim's authenticated session — deliverable via email, chat, or a malicious ad/page. Unlike the stored case, this requires the victim to click a crafted link.

Fix: Same as Finding 2 — context-aware output encoding (`htmlspecialchars`) at every render point, plus a restrictive CSP.

High-security result: Script rendered as inert text; no alert.

4.4 Finding 4 — CSRF (High, CVSS 8.0)

Module: `vulnerabilities/csrf/` | **Security level tested:** Low

Attack page (hosted locally with `python3 -m http.server 8000`):

```html
<html>
  <body onload="document.forms[0].submit()">
    <form action="http://localhost/dvwa/vulnerabilities/csrf/" method="GET">
      <input type="hidden" name="password_new" value="hacked123">
      <input type="hidden" name="password_conf" value="hacked123">
      <input type="hidden" name="Change" value="Change">
    </form>
  </body>
</html>
```

Technique: The password-change endpoint trusts the session cookie alone with no unpredictable per-request token, so a cross-origin auto-submitting form silently changes the victim's password while they are logged in.

`[INSERT SCREENSHOT: Burp Suite captured/modified login request + password successfully changed to hacked123]`

Impact: Silent account takeover / lockout; the same pattern applies to any other state-changing action protected only by ambient cookies.

Fix:

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST'
    || !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'] ?? '')) {
    http_response_code(403);
    exit('CSRF token validation failed');
}
```

Use per-session CSRF tokens on every state-changing form, require POST, and set cookies with `Secure; HttpOnly; SameSite=Lax` (or `Strict` where workflow allows).

High-security result: DVWA's session-bound `user_token` is required; replay without a current token is rejected and the password is not changed.

4.5 Finding 5 — OS Command Injection (Critical, CVSS 9.8)

Module: `vulnerabilities/exec/` | **Security level tested:** Low

Payloads:

```
127.0.0.1 && cat /etc/passwd
127.0.0.1 && whoami
```

`[INSERT SCREENSHOT: /etc/passwd contents returned after the ping output]` `[INSERT SCREENSHOT: whoami returning www-data]`

Impact: Arbitrary OS command execution as the web-server account (`www-data`) — local file disclosure, network reconnaissance from the server, and a foothold for lateral movement or privilege escalation.

Vulnerable pattern → Fix:

```
// Vulnerable
$output = shell_exec('ping -c 4 ' . $_GET['ip']);

// Fixed
$target = $_POST['ip'] ?? '';
if (!filter_var($target, FILTER_VALIDATE_IP)) {
    http_response_code(400);
    exit('Invalid IP address');
}
$process = new Symfony\Component\Process\Process(['ping', '-c', '4', $target]);
$process->mustRun();
echo htmlspecialchars($process->getOutput(), ENT_QUOTES, 'UTF-8');
```

Avoid shell invocation entirely where possible; validate input as a strict allowlist (a real IP address) before any system call; run the web process under least privilege.

High-security result: Not separately captured in this test pass — flagged for re-verification (see §8.2 limitations and remediation checklist).

4.6 Finding 6 — Unrestricted File Upload → Web Shell RCE (Critical, CVSS 9.8)

Module: DVWA File Upload (`hackable/uploads/`) | **Security level tested:** Low

Steps: 1. Upload `shell.php` — DVWA confirms: `../../hackable/uploads/shell.php successfully uploaded!` 2. Request: `http://localhost/dvwa/hackable/uploads/shell.php?cmd=id` 3. Response:

`uid=33(www-data) gid=33(www-data) groups=33(www-data)`

`[INSERT SCREENSHOT: shell.php upload success message]` `[INSERT SCREENSHOT: shell.php?cmd=id executing and returning the www-data identity]`

Impact: Full remote code execution as the web-server account — the most severe finding in this assessment. From here an attacker can read application source/config/secrets, deface content, pivot to internal services, or attempt privilege escalation.

Vulnerable pattern → Fix:

```php
// Vulnerable
$destination = '../../hackable/uploads/' . $_FILES['uploaded']['name'];
move_uploaded_file($_FILES['uploaded']['tmp_name'], $destination);

// Fixed
$allowed = ['image/jpeg' => 'jpg', 'image/png' => 'png'];
$mime = (new finfo(FILEINFO_MIME_TYPE))->file($_FILES['uploaded']['tmp_name']);
if (!isset($allowed[$mime])) { http_response_code(415); exit('Unsupported file type'); }
$safeName = bin2hex(random_bytes(16)) . '.' . $allowed[$mime];
$destination = '/srv/app-data/uploads/' . $safeName; // outside the web root
move_uploaded_file($_FILES['uploaded']['tmp_name'], $destination);
```

Also disable script execution in the upload directory (Apache `.htaccess`: `Options -ExecCGI`, deny `.php*` handlers), store uploads outside the document root, and serve downloads through a non-executable handler.

High-security result: Not separately captured in this test pass — flagged for re-verification (see §8.2 limitations and remediation checklist).

4.7 Supporting Evidence — Burp Suite Request Interception

`[INSERT SCREENSHOT: Burp Proxy — captured login request]` `[INSERT SCREENSHOT: Burp Proxy — modified request before forwarding]` `[INSERT SCREENSHOT: Resulting application state after the modified request]`

These captures document testing methodology (manual request tampering) and support Finding 4 (CSRF); they are not an independent vulnerability on their own.

5. Mitigation Recommendations — Consolidated Checklist

- [ ] Replace all SQL string concatenation with prepared statements / parameterized queries.
- [ ] Hash passwords with Argon2id, bcrypt, or scrypt, each with a unique salt — never unsalted fast hashes.
- [ ] Encode output for the correct context (HTML, attribute, URL, JS, CSS) at every render point.
- [ ] Deploy a restrictive Content-Security-Policy; avoid inline scripts.
- [ ] Add per-session CSRF tokens validated server-side on every state-changing action; require POST.
- [ ] Set cookies with `Secure`, `HttpOnly`, and an appropriate `SameSite` value.
- [ ] Never pass user input to a shell; use safe process APIs with strict allowlisting.
- [ ] Run the web server process under a least-privilege, unprivileged account.
- [ ] Validate uploads by MIME/content-signature (not just extension); generate server-side filenames; store outside the web root; disable script execution in the upload directory.
- [ ] Add regression tests that replay every payload in this report and assert no exploit result is returned.
- [ ] Keep DVWA and other intentionally vulnerable applications fully isolated from any production network.

6. Incident Response Report (Simulated)

Scenario: The Unrestricted File Upload finding (§4.6) is used to simulate a realistic incident: an attacker uploads `shell.php` to the DVWA upload directory and uses it to execute commands as the web server.

6.1 Detection

- The simulated SOC analyst reviews Apache access logs (`/var/log/apache2/access.log`) and identifies two anomalous indicators:
  - A `POST` request to the file-upload endpoint with a `.php` attachment, which should never be accepted by a legitimate image-upload form.
  - Repeated `GET` requests to `/dvwa/hackable/uploads/shell.php?cmd=...` — a `.php` file being *executed* from an "uploads" directory is a strong indicator of compromise.
- **Indicators of Compromise (IOCs):**
  - File: `hackable/uploads/shell.php`
  - URL pattern: `?cmd=` parameter against an upload-directory PHP file
  - Source IP / timestamp of the uploading and executing requests (captured from the access log)

6.2 Containment

1. Isolate the affected VM — disable or restrict its network interface to stop further attacker interaction.
2. Suspend the web server process (or block the specific source IP at the host firewall) to halt further command execution through the shell.
3. Preserve volatile evidence (running processes, open connections, current log state) before any remediation, for later root-cause analysis.

6.3 Eradication

1. Delete the malicious file: `hackable/uploads/shell.php`.
2. Search the entire upload directory and web root for other dropped files with unexpected extensions or recent modification timestamps.
3. Reset any credentials or session tokens that may have been exposed while the web-shell was active (the attacker ran as `www-data`, so check for any secrets that account could read).
4. Patch the root cause per §4.6's fix (MIME/signature validation, non-executable upload storage).

6.4 Recovery

1. Restore the DVWA application from a known-clean snapshot/backup.
2. Re-deploy with the upload-validation and non-executable-directory fixes applied.
3. Re-run the original payload (`shell.php` upload + `?cmd=id`) to confirm the fix holds before returning the system to service.
4. Resume normal monitoring.

6.5 Post-Incident Report Summary

Field	Detail
Incident	Unauthorized PHP web-shell upload and remote command execution
Root cause	Upload endpoint trusted client-supplied filename/extension and stored the file inside an

	executable, web-accessible directory
Detection method	Apache access-log review — anomalous `.php` upload + repeated `?cmd=` execution requests
Containment action	Network isolation of the VM; web process suspended
Eradication action	Malicious file removed; directory swept for additional artifacts
Recovery action	Clean restore + validated fix re-deployed
Lessons learned	Enforce upload allowlisting and non-executable storage by default on *every* upload feature, not only after an incident; add log-based alerting for `.php` requests inside upload paths
Recommended follow-up controls	Web Application Firewall (WAF) rule blocking script execution in upload paths; centralized log alerting; scheduled re-testing of this control

7. Conclusion

This assessment confirmed six exploitable vulnerabilities in DVWA at its Low security setting, spanning injection (SQL and OS command), broken access/session trust (CSRF), and improper output handling (stored and reflected XSS), plus a critical unrestricted file-upload flaw leading to full remote code execution. Each of these maps directly to a well-known OWASP Top 10 category, and each was confirmed to be blocked (for the four findings re-tested) once DVWA's High security mitigations — prepared statements, output encoding, and CSRF tokens — were enabled.

The exercise reinforces a consistent theme across all six findings: **trusting client-supplied input without validating, encoding, or parameterizing it at the point of use** is the common root cause. The fixes are equally consistent — parameterized queries, context-aware output encoding, per-session CSRF tokens, strict input validation before any system call, and allowlist-based, non-executable file storage.

The simulated incident-response walkthrough (Section 6) demonstrates that even a successfully exploited critical vulnerability can be detected from standard web server logs and contained/eradicated using a conventional IR lifecycle, provided logging and monitoring are in place beforehand.

Recommendation: before any similar application goes to production, re-run this same test suite against it, and add the regression tests listed in Section 5 to CI so that a reintroduced vulnerability is caught automatically rather than discovered through a later penetration test.

8. Appendix

8.1 Verification Matrix

Test	Low-security result	High-security result	Evidence status
SQL UNION payload	Usernames + hashes returned	No injected rows returned	Screenshot (Low)
Stored XSS script	Alert executes on page load	Rendered as literal text	Screenshot (Low)
Reflected XSS URL	Alert executes immediately	Encoded / no execution	Screenshot (Low)
CSRF auto-submit	Password changed silently	Requires valid session token	Screenshot (Low)
Command injection	`/etc/passwd` + identity returned	Not re-verified this pass	Screenshot (Low)
File-upload web shell	`shell.php` executes `id`	Not re-verified this pass	Screenshot (Low)

8.2 Evidence Limitations

- High-security “blocked” screenshots for Command Injection and File Upload were not captured in this test pass — these should be added before final submission to fully satisfy the “exact same test, High security” verification step.
- All `[INSERT SCREENSHOT: ...]` placeholders in Section 4 must be replaced with the actual PNG captures from the `evidence/` folder referenced in the project’s GitHub repository before the PDF is submitted.

8.3 Reconnaissance Output (example commands used)

```
nmap -sV -p- <DVWA-IP>
nikto -h http://<DVWA-IP>
sqlmap -u "http://<DVWA-IP>/vulnerabilities/sqli/?id=1&Submit=Submit#" \
       --cookie="PHPSESSID=<value>; security=low" --dump
```

8.4 Deliverables Cross-Reference

- Capstone Project Report (this PDF).
- GitHub repository: scripts, payload lists, and this report’s Markdown source.
- 12-minute final presentation video (walkthrough of objective → live exploit demo → mitigation → IR simulation → conclusion).

Source material: compiled and restructured from the project’s DVWA testing notes and the referenced source report/screenshots.